

Engineering Roadmap & Methodology — the system for building software (validated against the standards)

What this document is: the definitive, **sourced** answer to "what is the complete process for *building* software, not just coding — and who says so." Every major claim below is cited to an authoritative body (IEEE / ISO) or to the practitioner who originated the named practice. This is the permanent "system" you adopt once and apply to every project.

Provenance: validated 2026-07-21 against a multi-source research pass (SWEBOK v4, the ISO/IEC/IEEE 12207/15288/29148/29119 family, DDD, C4, Microsoft SDL, "Software Engineering at Google", Amazon "Working Backwards", UC Berkeley CS169A). 25 claims verified 3-of-3, 0 refuted. Sources listed at the end. Honest scope-gaps also listed — I mark what was **not** verified so you never mistake a gap for a fact.

The one-sentence answer to "according to whom?"

There is no single authority that publishes one ordered checklist. The closest things to *the* definition of software engineering are two standards bodies:

- **SWEBOK v4** — the *Guide to the Software Engineering Body of Knowledge*, published by the **IEEE Computer Society** (v4 released **October 2024**). It defines the 18 **Knowledge Areas** (Requirements, Architecture, Design, Construction, Testing, Operations, Maintenance, Security, ...).
- **The ISO/IEC/IEEE life-cycle family** — **12207** (software life-cycle processes), **15288** (systems), **29148** (requirements engineering), **29119** (testing), **42010** (architecture description).

These define **what the work is and how the pieces depend on each other** — but they deliberately **do not mandate a rigid temporal order**. 12207/15288/29148 explicitly say the processes are applied "*iteratively, recursively and concurrently*", and SWEBOK adds "*there are no strict boundaries between stages*" and "*not all stages are found in every process*." So "the correct order" is a **logical dependency order** (requirements feed design; design traces back to requirements), not a waterfall. Everything below is that logical order — you iterate over it.

Named methods you have heard of (**DDD, C4, ADRs, RUP, Scrum**) are **not** competitors to the standards — they *plug into* this skeleton. DDD and C4 are *design techniques*; ADRs are *how you record architecture decisions*; RUP/Scrum are *how you schedule the iterations*. The standards say *what*; the methods say *how*.

The two ideas that make the order make sense

Idea 1 — Problem space before solution space

First understand *what the business needs and why* (no technology). Only then design *how*. **ISO/IEC/IEEE 29148** encodes this: its three requirements processes — **Business/Mission Analysis** → **Stakeholder Needs & Requirements** → **System/Software Requirements** — produce a *baselined, traceable* requirement set that **"flows into the Architecture and Design processes."** Requirements before design is not a preference; it is written into the standard.

Idea 2 — The order is a *dependency order*

SWEBOK v4 defines design itself as an ordered chain: **architectural** → **high-level** → **detailed**, where "each stage, when present, creates an obligation upon the next." High-level design happens "within the envelope established by the architecture"; detailed design "within the constraints established by the high-level design." You literally cannot do a stage well until the one it depends on exists. That is why the pipeline has an order at all.

Idea 3 — It is **ONE model**, described through many **VIEWS**, each more precise than the last

This is the idea that ties the whole pipeline together. **The steps are not building different things. They describe the SAME things (the domain concepts) over and over — each time from a new angle, each time adding the precision that angle needs.** First you *discover and name* the things (words). Then you give them *structure*. Then *behaviour*. Then *storage*. Same things, sharper each pass. (In UML this is why diagrams split into "structural" vs "behavioural" — they are different views of one model. It is also the whole point of Kruchten's 4+1 views.)

Worked example — the **Booking**, one single concept seen through every view:

Step / view	What it says about Booking	The angle it adds
Domain (step 1)	"A Booking is one customer's checkout holding 1..n reservations."	NAME + meaning (just words)
Aggregate / class (step 3)	Booking <i>owns</i> its Reservations; carries code, customer, obligations.	STRUCTURE — what it is made of
State machine (step 7)	Booking: Pending → Confirmed → Completed / Cancelled.	BEHAVIOUR over time
Sequence (step 8)	On a payment webhook: Gateway → Booking → Reservation flip.	INTERACTION — how it collaborates
ERD (step 10)	<code>booking(id, code, customer_id, status, ...)</code> + constraints.	STORAGE — final executable form

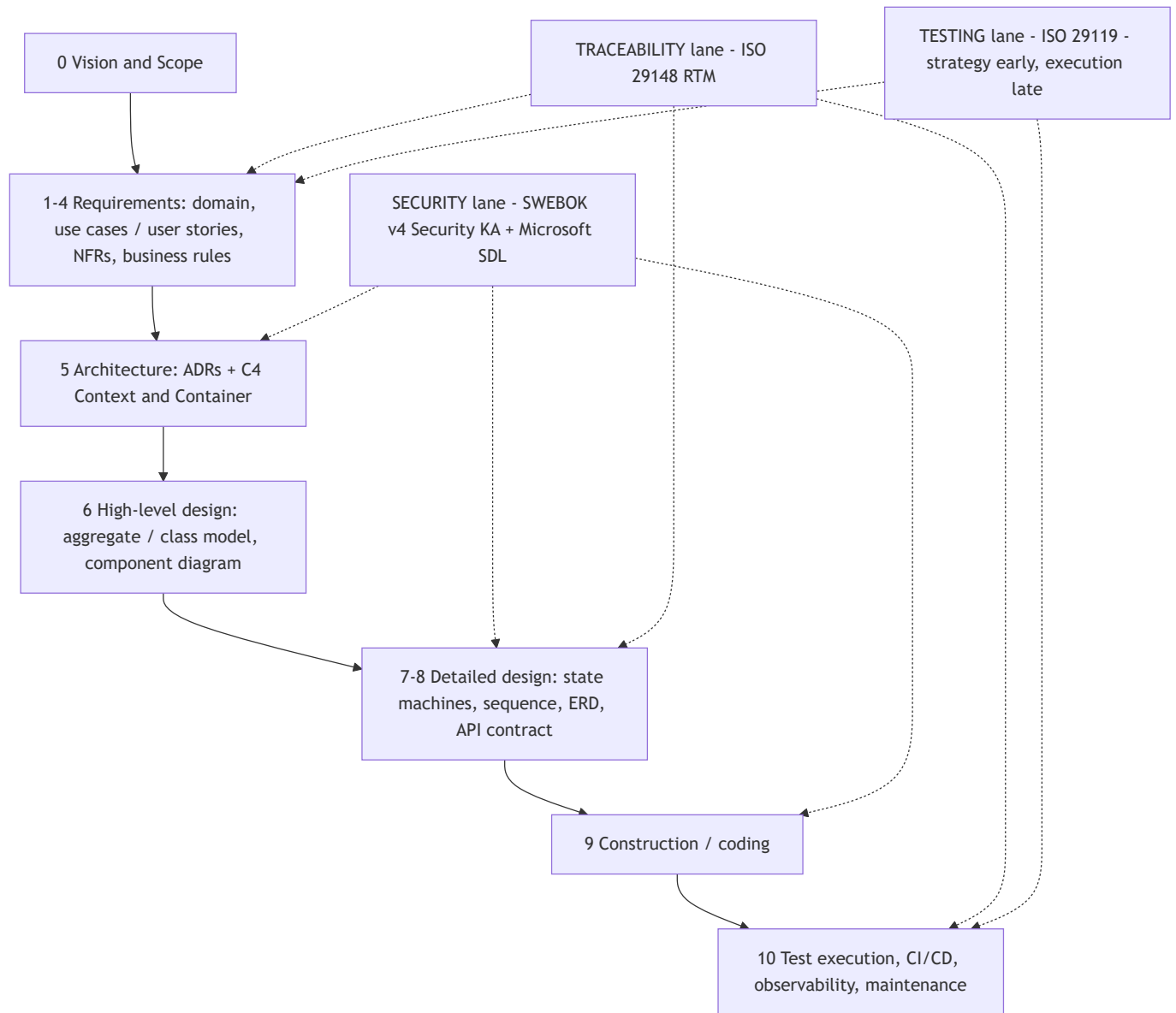
Notice the punchline: the **ERD invents almost nothing new**. The `status` column comes from the state machine. The `customer_id` link comes from the class model. The meaning comes from the domain. **The ERD is the persistence projection of everything the earlier views already established** — which is exactly why it comes late (it depends on all of them) and why, once the earlier views exist, deriving it is mostly mechanical. "We already kinda had it" is *correct*: the late steps **consolidate**; they do not re-discover.

This also explains why jumping straight to sequence/state diagrams (as the old design did) fails: a view is only as trustworthy as the model beneath it. Draw the interaction view before the structure and the complete list of things exist, and you are describing things that are not yet fully defined — which is precisely why the old design "felt random and incomplete."

The pipeline — with the two things my first draft got wrong made visible

The big correction from the research: **security and testing are not single late steps — they are cross-cutting lanes that touch every stage**. Same for **traceability**. Here is the corrected shape:

The picture above is a rendered image, so it shows as a diagram in any viewer. The same diagram in **Mermaid source** is below — copy it into draw.io if you want to edit it.



Phase	Artifact	What it is · UML / notation	Who defines it
0 Framing	Vision & Scope	Problem, goals, in/out of scope. <i>Notation: prose (a "design doc" / PR-FAQ).</i>	Amazon <i>Working Backwards</i> (PR/FAQ written first , before code); Google gating design doc
1 Problem space	Business Domain model	Ubiquitous language, actors, entities, rules. <i>Notation: domain model, glossary.</i>	DDD (Evans) — Ubiquitous Language ; SWEBOOK "Requirements" KA
1	Use cases or user stories	The complete list of what each actor can do. <i>Notation: UML use-case diagram, or agile user stories.</i>	ISO 29148 (requirements); Berkeley CS169A teaches user stories , not UML — both are legitimate
1	Non-Functional Requirements	Scale, security, RTL, uptime — the quality attributes. <i>Notation: NFR list / spec.</i>	ISO 29148 "SRS = functions and performance, design constraints, attributes"

Phase	Artifact	What it is · UML / notation	Who defines it
1	Business Rules & Requirements spec	The formal, ID'd rules; the baselined requirement set. <i>Notation: SRS/SyRS; each rule Singular, Unambiguous, Verifiable.</i>	ISO 29148 — the normative " construct of a good requirement "
2 Architecture	ADRs	Each nontrivial decision + its rationale + trade-offs.	SWEBOK v4: architecture = " <i>a network of decisions ... documented with rationale</i> "; ADR format = Michael Nygard
2	C4 Context + Container	System in its environment; the major deployable pieces. <i>Notation: C4 levels 1–2 = SWEBOK component + deployment views.</i>	Simon Brown (C4); SWEBOK v4 "Software Architecture" KA (its own chapter in v4)
3 High-level design	Aggregate / domain model	Entities, value objects, aggregates , invariants. <i>Notation: UML class / object diagram.</i>	DDD tactical (Evans); SWEBOK "Structural design descriptions"
3	Component diagram	How the containers break into components. <i>Notation: C4 Component = UML component diagram.</i>	SWEBOK v4 (structural); C4 level 3
4 Detailed design	Lifecycles / state machines	Canonical states + transition tables. <i>Notation: UML state-machine diagram.</i>	SWEBOK "Behavioral design descriptions"
4	Sequence / flow diagrams	Who calls whom, in order, per use case. <i>Notation: UML sequence / communication diagram, activity diagram.</i>	SWEBOK "Interaction diagrams"
4	Data model / ERD	Persistence schema with constraints. <i>Notation: ERD (SWEBOK lists it under structural).</i>	SWEBOK v4 structural design descriptions
4	API contract	The exact service interface. <i>Notation: OpenAPI.</i>	(practice standard)
Cross-cutting	Security & threat model	STRIDE threat model at design ; secure coding; security tests.	SWEBOK v4 Software Security KA (" <i>consider every life-cycle stage</i> "); Microsoft SDL (threat modeling = practice #3, Design)
Cross-cutting	Traceability (RTM)	Links requirement ↔ design ↔ test, both directions.	ISO 29148 — Requirements Traceability Matrix is a named artifact
Cross-cutting	Test strategy	Strategy defined early (shift-left); execution late.	ISO/IEC/IEEE 29119-3:2021 (IEEE 829 is superseded — do not cite it)
5 Delivery	Build plan → construction	Ordered milestones (CORE/MIN/POST), then code.	SWEBOK "Construction" KA
5	CI/CD, observability, maintenance	Pipeline, monitoring, run-and-evolve.	SWEBOK v4 Software Engineering Operations KA (new in v4) + Maintenance KA

Where UML and OOP fit (the gap you caught)

You were right that UML wasn't showing — but it is **not a step**. Per **SWEBOK v4**, "*UML is used in **all** design stages, from architectural through detailed design.*" It is the **notation** you draw *inside* the design steps:

- **Use-case diagram** → step 1 (requirements).
- **Class / object diagram** → step 3 (the aggregate/domain model — this is the **OOP** structural view).
- **Component diagram + deployment diagram** → steps 2–3 (architecture / high-level). (*These two were missing from my first table — now added.*)
- **State-machine diagram** → step 4 (lifecycles).
- **Sequence / communication + activity diagram** → step 4 (interactions/flows).

Where OOP is "grounded": DDD's **Model-Driven Design** ties one domain model directly to the code, which Evans says "*usually requires object-oriented programming.*" So the aggregate/class model (step 3) **is** your object-oriented design. You do not need a separate "OOP step" — OOAD lives in steps 1→3.

Where DDD, C4, ADRs, threat modeling fit

- **DDD (Evans):** *strategic* design (Ubiquitous Language, **Bounded Contexts**, context maps) sits at the domain/architecture level (steps 1–2); *tactical* design (entities, value objects, **aggregates**) is the high-level class model (step 3).
 - **C4 (Brown):** Context + Container = architecture (step 2); Component = high-level (step 3); Code (optional, rarely drawn) = detailed.
 - **ADRs (Nygard):** the concrete form of SWEBOK's "*network of documented decisions with rationale*" — one small file per nontrivial decision, at step 2 and whenever a decision is made.
 - **Threat modeling (STRIDE / Microsoft SDL):** **at the design stage**, on your C4 container diagram — *not* a late one-off. Security is a lane, not a step.
-

How to design the components (the method)

Naming the pieces — services, repositories, controllers — is where people **cargo-cult**: they copy "services + repositories" without knowing *why*. Here is the actual method, so the structure is **derived, not guessed**. It works for any system, at any scope.

The one goal: keep the system **easy to understand and cheap to change**. Complexity is the enemy; every choice is judged against it. (*Ousterhout, "A Philosophy of Software Design."*)

The two forces (the measuring stick for any structure):

- **High cohesion** — one piece, one clear job.
- **Low coupling** — pieces touch as little as possible, through small interfaces. (*Constantine & Yourdon, structured design.*)

The steps — a logical order you iterate over:

1. **Start from behaviour + language, not boxes.** The jobs (verbs = use cases) and the things (nouns = the domain's own words). (*DDD — model from the domain.*)
2. **Group by "reason to change."** Same reason → together; different reasons → apart. This is *why* domain / persistence / API are separate: they change for different reasons. (*Single Responsibility Principle — R. C. Martin: "one reason to change."*)
3. **Assign each responsibility to its natural owner** (*GRASP — Larman*): the one **with the data** does the job (Information Expert); the one that **owns X creates X** (Creator); a job **spanning several objects** gets a

coordinator — that coordinator **is an application service** (Controller). Break ties by choosing the option with lower coupling / higher cohesion.

4. **Point dependencies at the domain.** The domain (rules) depends on **nothing**; everything depends on it. Anything external or swappable sits behind a **port** (interface) in the domain, with an **adapter** outside it. (*Dependency Inversion — SOLID; Hexagonal / Ports-and-Adapters — Cockburn; Clean Architecture — Martin.*)
5. **Draw the boundary around what's likely to change.** Wrap each volatile thing (payment provider, pricing formula, UI) behind an interface so a future change stays **inside** one piece. (*Information hiding — David Parnas, 1972.*)
6. **Test, then redraw if it fails.** For each piece: **one-sentence job?** (cohesion) · **change its insides without touching others?** (coupling) · **understandable without opening five other files?** If any check fails, the boundary is wrong — redraw.

Why the pattern names are not dogma: they *fall out* of the steps. A **service** = the coordinator for a multi-object job (step 3). A **repository** = the isolated home for the storage-reason-to-change (step 2). A **port/adapter** = the swappable external thing (steps 4–5). If a different shape scored better on cohesion + coupling, you'd use it and be right. **The principles are the authority; the patterns are just the answers those principles keep producing.**

Sources for this method: Constantine & Yourdon (coupling & cohesion); R. C. Martin, *Clean Architecture* / SOLID; Craig Larman, *Applying UML and Patterns* (GRASP, responsibility assignment); David Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules" (1972, information hiding); Alistair Cockburn (Hexagonal / Ports-and-Adapters); Eric Evans (DDD); John Ousterhout, *A Philosophy of Software Design*.

The 6 corrections the research made to my first draft (with who says so)

My first 14-step table was **broadly sound** — the research confirmed that putting requirements (steps 0–4) before architecture matches ISO 29148's logical precedence, and that **ERD after lifecycles is defensible, not an error** (SWEBOK: "no strict boundaries" — deriving the ERD from the aggregate model is reasonable). But it fixed six things:

1. **Security was mis-placed as a late step 9.** → Make it a **cross-cutting lane**; run the threat model at the architecture/design cluster. (*SWEBOK v4 Software Security KA; Microsoft SDL practice #3.*)
 2. **Missing UML structural diagrams.** → Add **component** and **deployment** diagrams; the aggregate model only covered the class view. (*SWEBOK v4 structural design descriptions.*)
 3. **C4 was lumped into one step.** → Decompose into explicit **Context / Container / Component** levels. (*c4model.com.*)
 4. **No traceability.** → Add a **Requirements Traceability Matrix** (or, for a solo dev, lightweight story↔test links). (*ISO 29148.*)
 5. **Test artifact mis-cited and too late.** → Cite **ISO/IEC/IEEE 29119-3:2021**, not the superseded IEEE 829; define the **test strategy early** (shift-left), execution stays late.
 6. **Ops/maintenance too thin.** → Add **CI/CD, observability, maintenance** explicitly. (*SWEBOK v4 added a Software Engineering Operations KA.*)
-

The LEAN SOLO version — what YOU actually do under deadline

The standards permit tailoring: SWEBOK says "*not all stages are found in every process.*" For a solo engineer under time pressure, **keep the high-signal artifacts, cut the ceremony:**

KEEP:

1. **One gating design doc** (Google one-pager / Amazon PR-FAQ-lite): goals, key decisions + trade-offs, security, storage/privacy. (*This is your Vision + light architecture in one file.*)
2. **Requirements as user stories + an explicit NFR list.** Apply 29148's quality checks informally — each item singular, unambiguous, verifiable.
3. **ADRs** for each nontrivial decision (cheap, high-value — this is what preserves *why*).
4. **C4 Context + Container** diagrams; add **one** Component diagram only for the most complex part; **skip the Code level.**
5. **Domain / aggregate model + ubiquitous language.** Bounded contexts only if the system is non-trivial.
6. **Behavioral diagrams only where the logic is hard** — a **state machine for the reservation lifecycle**, one or two **sequence diagrams** for the booking/payment flow. Not for everything.
7. **ERD** derived from the aggregates.
8. **API contract** (OpenAPI).
9. **A lightweight threat model at design time** — STRIDE on the container diagram. Not a separate phase.
10. **A one-page test strategy** + automated tests + **story↔test traceability** (not a heavyweight RTM).
11. **CI/CD + basic observability.**

CUT / DEFER:

- Separate StRS / SyRS / SRS documents → collapse into the one design doc.
- Exhaustive UML (object, communication, timing, composite-structure, package diagrams).
- The full ISO 29119 document set; a formal heavyweight RTM.
- Full RUP discipline ceremony.

This keeps the standards' **obligation chain** (requirements → architecture → high-level → detailed → test) while minimizing the number of documents — which is exactly what Google's single gating design doc and Berkeley's agile course both do in practice.

Honest scope-gaps (what the research did NOT verify — do not treat these as settled)

The research pass produced **no independently verified claim** for these, even though they were in scope. I flag them so you don't mistake my general knowledge for a sourced fact:

- **Kruchten's 4+1 view model** (logical / process / development / physical / scenarios). (*I referenced it earlier as the UML-to-view map; the authoritative UML placement actually comes straight from SWEBOK v4, which is stronger. 4+1 remains a useful complementary lens — just not verified here.*)
- **arc42** (the 12-section architecture template) and where it sits relative to C4.
- **ISO/IEC/IEEE 42010** (architecture description: viewpoints/stakeholders/concerns) and **IEEE 1016** (software design descriptions) — what extra named artifacts they mandate.
- The detailed **RUP** phase-and-discipline mapping onto a lean solo flow.

If we ever want these nailed down with citations, that is a focused follow-up research pass.

Key sources (citable)

- **SWEBOK v4** — IEEE Computer Society, 2024 — ieeecs-media.computer.org/media/education/swebok/swebok-v4.pdf
- **ISO/IEC/IEEE 29148:2018** (requirements engineering) — iso.org/standard/72089.html
- **ISO/IEC/IEEE 29119-3:2021** (test documentation) — iso.org/standard/79429.html
- **ISO/IEC/IEEE 12207** (software life-cycle processes)
- **Domain-Driven Design**, Eric Evans, 2003 (Ubiquitous Language, Bounded Context, aggregates)
- **The C4 model**, Simon Brown — c4model.com
- **Architecture Decision Records**, Michael Nygard — "Documenting Architecture Decisions"
- **Microsoft Security Development Lifecycle (SDL)** — microsoft.com/securityengineering/sdl
- **Software Engineering at Google** (design-doc culture) — abseil.io/resources/swe-book/html/ch10.html
- **Amazon "Working Backwards" PR/FAQ** — Bryar & Carr — workingbackwards.com
- **UC Berkeley CS169A** (Fox & Patterson, agile/XP, user stories)

Where we are right now

The **live tracker** is [../docs3/README.md](#) (the project was rebuilt into docs3/). Snapshot as of 2026-07-26:

Stage	Artifact	Status
0 Framing	Vision & Scope	✓ docs3/0-framing/vision-and-scope.md
1 Problem space	Business domain · use cases · NFRs · business rules	✓ docs3/1-problem-space/
1 Problem space	Edge-case research (real-world policy catalog)	✓ docs3/1-problem-space/edge-case-research.md
2 Architecture	Overview (C4 Context/Container) + 7 ADRs	✓ docs3/2-architecture/
3 High-level	Aggregate / class model	✓ docs3/3-high-level-design/aggregate-model.md
3 High-level	Component design	🟡 scaffold — designing now, together (the method above)
4 Detailed	Lifecycles (state machines)	✓ docs3/4-detailed-design/lifecycles.md
4 Detailed	Sequence flows · ERD · API contract	🟡 next, after the component design
—	Perf & data plan (indexes, caching, retention/cleanup)	🟡 offered; delivery stage
5 Delivery	CI/CD, ops, build plan	🟡 pending

Immediate next step: design the **components** (stage 3) module by module, foundation → up, using the method above — then the **sequence flows** on top of them.